# AnonyMus — Comprehensive Technical Plan

**Audience:** Internal Development Team **Purpose:** Architecture analysis, security audit, and implementation-ready plan for next development cycle **Scope:** Zero-knowledge model hardening, disappearing messages, Android app improvements **Guiding Principles:** Privacy, Security, Accessibility, Ease-of-Use **Date:** June 2026

---

## Table of Contents



---

## 1. True Architecture Assessment

### 1.1 What the Codebase Actually Is

AnonyMus is a **stateless, zero-knowledge, WebSocket-relayed encrypted chat system**. After thorough analysis of every source file, the actual architecture is as follows:

**Server (`server.py`) — 324 lines:**

- Flask + Flask-SocketIO with `eventlet` async worker
- HTTP routes: `/` (redirect), `/chat` (session-gated), `/register`, `/login`, `/logout`
- WebSocket events: `connect`, `create_queue`, `push_queue`, `disconnect`
- The server acts as a **pure relay** — it joins SocketIO rooms and emits payloads to them. It does **not** store, read, or log message content at any point.
- Rate limiting: `flask-limiter` on HTTP endpoints (5/min register, 10/min login, 200/day 50/hr global), plus custom in-memory per-socket rate limiting

(`is_rate_limited()`) for WebSocket events (5/10s for queue creation, 10/1s for message push)

- Security headers: HSTS, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy no-referrer, Permissions-Policy (camera/mic/geo blocked), and a **Content-Security-Policy** that restricts script-src to `'self'` and `cdn.socket.io`, allows `wss:` and `ws:` in connect-src, and blocks all object/embed sources
- Session management: server-side Flask sessions with `SESSION_COOKIE_SECURE`, `HTTPONLY`, `SAMESITE=Strict`
- Session fixation protection: `session.clear()` before setting new session on login
- Request size limit: 1MB via `MAX_CONTENT_LENGTH`
- SSL: self-signed cert generation with SHA-256, RSA-2048, includes SAN for localhost + local IP
- Payload size limit on WebSocket: 100KB hard cap in `handle_push_queue`
- CORS: configurable via `CORS_ORIGINS` env var, defaults to `*` (flagged as risk below)
- Redis support: optional `REDIS_URL` for multi-worker SocketIO message queue

**Database (`database.py`) — 103 lines:**

- Single `users` table: `username TEXT PRIMARY KEY, password_hash TEXT NOT NULL`
- bcrypt password hashing with constant-time dummy hash for non-existent users (timing attack mitigation)
- PostgreSQL support via `DATABASE_URL` env var (falls back to SQLite)
- Case-insensitive username lookup (`username.lower()`)
- Username max 50 chars, password max 128 chars (enforced in server.py, not database.py)
- **No message storage. No room table. No metadata table. No logging table.**

**Web Crypto (`crypto.js`) — 212 lines:**

- ECDH P-256 key pair generation via WebCrypto API
- HKDF-SHA256 key derivation with domain-separated labels: `AnonyMus-Client-To-Server-Key` and `AnonyMus-Server-To-Client-Key`
- Role assignment: lexicographic comparison of Base64 public keys determines Alice/Bob roles
- AES-256-GCM encryption with 12-byte random IV, 5-byte AAD (role char + 32-bit big-endian sequence number)
- Length-prefixed random padding: plaintext length stored as 32-bit big-endian, padded to 512-byte blocks with random bytes
- Safety number: SHA-256 of sorted concatenated public keys, displayed as 8-char hex

chunks separated by dashes

- Base64 encoding for all binary data transport

**Web Chat Client (`chat.js`) — 459 lines:**

- Socket.IO client with WebSocket-only transport
- Session state machine: keypair generation, queue creation, invite link generation (URL hash fragment with queue ID + public key), QR code generation
- Handshake protocol: invitee derives shared keys and sends `{type: "handshake", reply_queue, public_key}` to host's queue
- Reconnection support: on reconnect, sends `queue_update` to inform peer of new queue ID
- Keep-alive ("heartbeat"): encrypted control messages at random 2-7 second intervals to obscure traffic patterns
- Screen blur on tab hidden, triple-Escape panic button (resets session, navigates to about:blank)
- Clipboard auto-clear: copied invite links are cleared from clipboard after 30 seconds
- Disappearing messages: client-side DOM removal via `setTimeout` based on dropdown-selected timer
- Clear cache button: sends encrypted `{action: "clear"}` control message, then resets session

**Web Auth (`login.js`) — 61 lines:**

- Simple fetch-based login/register with JSON payloads
- Client-side form validation (empty check only)
- No client-side password policy enforcement

**Android App (Kotlin, Jetpack Compose):**

- `CryptoUtils.kt`: ECDH P-256 via standard Java JCE (not Tink as originally planned), manual HKDF using HMAC-SHA256, AES-256-GCM with identical AAD construction and padding scheme to web client
- `ChatManager.kt`: singleton Socket.IO client, TOFU certificate pinning (SHA-256 SPKI hash), OkHttp cookie jar for session persistence, RAM sterilization on reset (zero-fills key byte arrays), forward secrecy (new keypair per connection), "psycho-historical static" traffic padding
- `NsdHelper.kt`: Android Network Service Discovery for LAN server discovery (`_http._tcp.` service type)
- `navigation.kt`: 4-screen flow: Config -> Auth -> Setup -> Chat
- `chat_screen.kt`: Compose UI with disappearing message timer dropdown

(Off/15s/60s), "Covert Mode" (disguises chat as calculator), "Obliviate" button (sends encrypted wipe command + local reset), "Infinity Snap" panic button (clears clipboard + resets + restarts app)

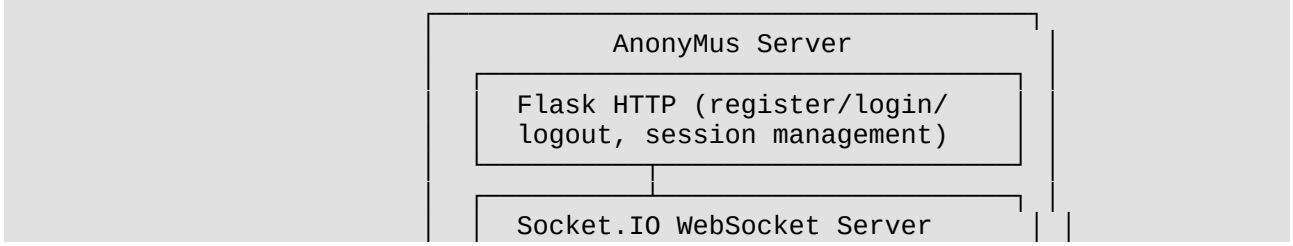
- `preferences_helper.kt`: stores host, port, session cookie, server cert fingerprint, device ID
- `auth_screen.kt`: login/register forms
- `config_screen.kt`: server host/port/trust-self-signed configuration

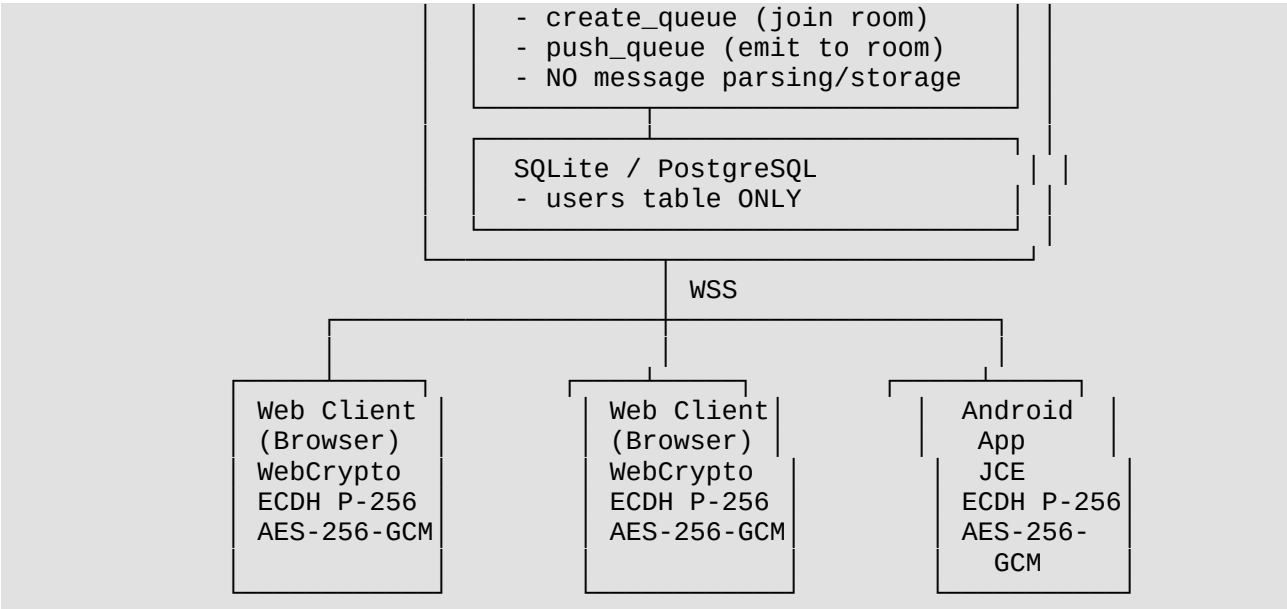
## 1.2 What the Original Plan Got Wrong

The original `Plan.md` contained several factual inaccuracies when compared against the actual codebase:

| **Claim in Original Plan** | **Actual Codebase Reality** |
|---|---|
| Server stores messages in a messages table | No message storage exists. Database has only a `users` table. Server is a pure relay. |
| CSP headers are missing | CSP is implemented in `set_security_headers()` with strict script-src, style-src, and connect-src policies. |
| No rate limiting exists | Both HTTP-level (flask-limiter) and WebSocket-level (custom `is_rate_limited()`) rate limiting are implemented. |
| Android uses Tink library | Android uses standard Java JCE (`KeyPairGenerator`, `KeyAgreement`, `Cipher`) — not Tink. |
| SQLite uses WAL mode | No WAL mode pragma is set. `database.py` uses default SQLite connection without journal mode configuration. |
| Server has no session fixation protection | `session.clear()` is called before setting new username session on login (line 169). |
| No transport encryption | Self-signed SSL with RSA-2048 + SHA-256, auto-generated on first run. |
| No timing attack mitigation | `DUMMY_HASH` constant ensures constant-time bcrypt comparison even for non-existent users. |

## 1.3 Architecture Diagram (Conceptual)






The server never sees plaintext. It only sees encrypted blobs routed through SocketIO rooms. Each "conversation" is two users sharing a pair of queue IDs (one per direction) through an out-of-band invite link.

---

# 2. Security Posture Deep-Dive

## 2.1 Current Strengths

**Cryptographic Protocol:**

- ECDH P-256 with HKDF-SHA256 key derivation is a sound, well-studied construction. The use of domain-separated labels (`Client-To-Server-Key` / `Server-To-Client-Key`) prevents key confusion attacks between the two directional keys.
- AES-256-GCM with 12-byte random IVs and 128-bit authentication tags provides both confidentiality and integrity. The 12-byte IV is the standard recommended length for GCM (avoids IV reuse risks inherent in longer IVs).
- The AAD construction (role byte + 32-bit sequence number) binds each ciphertext to its directional context and ordering, preventing cut-and-paste and replay attacks within a session.
- Length-prefixed random padding to 512-byte blocks is an excellent traffic analysis countermeasure — all ciphertexts are the same size regardless of plaintext length (up to 508 bytes), making it impossible to infer message length from wire traffic.
- Forward secrecy: new ECDH keypairs are generated per connection (Android) or per session (web), meaning compromise of long-term keys does not reveal past session keys.
- Safety numbers enable manual verification of peer identity, similar to Signal's safety number concept.

**Operational Security:**

- Timing attack mitigation on login via constant-time dummy bcrypt comparison.
- Session cookies are Secure, HttpOnly, SameSite=Strict — immune to XSS theft, CSRF, and subdomain cookie attacks.
- Session fixation protection via `session.clear()` before new session creation.
- TOFU certificate pinning on Android (SHA-256 SPKI hash stored in SharedPreferences).
- RAM sterilization: Android zero-fills key byte arrays on `resetClient()`.
- Screen blur on tab hidden (web), Covert Mode calculator disguise (Android).
- Clipboard auto-clear after 30 seconds (web), clipboard clear on Infinity Snap (Android).
- Triple-Escape panic button destroys all in-memory state immediately.

**Network Security:**

- WebSocket-only transport (no long-polling fallback that could leak cookies via query params).
- Self-signed SSL with auto-generation ensures encrypted transport even without a CA-signed cert.
- HSTS with `includeSubDomains` enforces HTTPS for 1 year.
- Referrer-Policy `no-referrer` prevents URL leakage to third parties.
- X-Frame-Options DENY prevents clickjacking.

## 2.2 Current Weaknesses & Gaps

**CRITICAL — No Password Policy Enforcement:** The server enforces maximum lengths (username 50, password 128) but imposes no minimum password strength requirements. Users can register with passwords like "a" or "123". While bcrypt slows brute-force attacks, a 1-character password has only ~95 possible values (printable ASCII), which bcrypt cannot protect against. A minimum password length of 8 characters with at least one character from three of four categories (uppercase, lowercase, digit, special) should be enforced server-side in the `/register` handler. Client-side validation in `login.js` should mirror this to provide immediate feedback, but server-side validation is the authoritative check.

**CRITICAL — WebSocket Authentication Bypass Risk:** The `handle_connect` handler (line 182-190) checks `if 'username' not in session: return False`. However, the SocketIO connection itself is established before this check returns. If the server uses a multi-process worker model without Redis, or if the session store is not shared, a race condition could allow a brief window where an unauthenticated socket emits events. Additionally, the `session` object in WebSocket handlers relies on the Flask session cookie being sent during the SocketIO handshake — if the cookie is missing or expired, the connection is rejected, but the error path could leak timing information.

**HIGH — CORS Defaults to Wildcard:** Line 82-84: `allowed_origins = os.environ.get("CORS_ORIGINS", "*").split(",")`. If `*` is in the list, it becomes `"*"` directly. This means any origin can make WebSocket connections, which is acceptable for a

local/LAN tool but dangerous if exposed to the internet. A malicious website could establish a WebSocket connection if the user's session cookie is available (mitigated by SameSite=Strict, but still a defense-in-depth gap).

**HIGH — In-Memory Rate Limiting is Per-Process:** The `socket_rate_limits` dictionary and its lock are in-process. If the server runs with multiple eventlet workers, each worker maintains its own rate limit state. An attacker could distribute requests across workers to bypass limits. This is partially mitigated by the `flask-limiter` HTTP rate limiter (which supports Redis backend), but WebSocket rate limiting lacks this support.

**HIGH — No Account Lockout or Brute-Force Protection Beyond Rate Limits:** Login is rate-limited to 10/minute, but there is no account lockout mechanism after N failed attempts. An attacker with patience could attempt 10 passwords per minute indefinitely. The bcrypt cost factor provides some slowdown, but a targeted attack on a known username is feasible.

**MEDIUM — SQLite Without WAL Mode:** `database.py` uses `sqlite3.connect(DB_FILE)` without setting `PRAGMA journal_mode=WAL`. In the default DELETE journal mode, readers block writers and vice versa. Under concurrent access (e.g., multiple registration attempts), this causes `sqlite3.OperationalError: database is locked`. WAL mode allows concurrent readers and a single writer without blocking.

**MEDIUM — No Input Validation on Username Characters:** The server enforces username length (max 50) and password length (max 128), but does not validate username character set. Usernames containing control characters, Unicode normalization variants, or SQL injection strings (mitigated by parameterized queries) could cause display issues or unexpected behavior.

**MEDIUM — Duplicate `const socket` Declaration in `chat.js`:** Lines 1-3 of `chat.js` contain `const socket = io({ transports: ['websocket'] });` declared twice. While JavaScript engines use the second declaration (or throw in strict mode), this is a code quality issue that could cause confusion or bugs during minification.

**MEDIUM — Android TOFU Without User Notification:** The Android TOFU implementation silently pins the first certificate seen (`Log.i(TAG, "TOFU: Pinned new server public key SPKI hash: $base64Hash")`). If a user connects to a compromised server on first use, the attacker's certificate is pinned permanently. There is no UI to show the fingerprint for manual verification, no warning on first connection, and no mechanism to reset the pin (short of clearing app data).

**MEDIUM — No WebSocket Message Authentication:** The server's `push_queue` handler trusts the `queue_id` from the client without verifying that the client is authorized to send to that specific queue. Any authenticated user can emit `push_queue` to any queue ID they know. Since queue IDs are UUIDs (128 bits of entropy), brute-force is infeasible, but if a queue ID is leaked (e.g., in logs, referer headers), any authenticated user can inject messages into that conversation.

**LOW — No CSP Nonce or Hash for Inline Scripts:** The CSP allows `'unsafe-inline'` in `style-src`. While scripts are restricted to `'self'` and the CDN, inline styles are permitted. This is a minor risk since style injection cannot execute JavaScript, but it could be used for UI redressing or data exfiltration via CSS.

**MEDIUM — `device_id` Sent but Not Used:** The Android client sends `device_id` in

registration and login payloads (line 271, 295 of `chat_manager.kt`), but the server's `/register` and `/login` handlers do not read or store it. This is dead code that creates a false impression of device binding. It should either be fully implemented (store device ID in a `devices` table, require re-authentication from unrecognized devices, enable per-device session revocation) or removed entirely to avoid confusion.

**LOW — No Proximity-Based Party Verification:** Safety numbers are computed and displayed but there is no QR-code-based verification flow (like Signal's scanning mechanism) to make verification user-friendly. Users must manually compare 32-character hex strings, which is impractical and rarely done in practice. Consider implementing a QR code that encodes both users' public key fingerprints, scannable by the other party's device camera to automatically confirm a match.

## 2.3 Threat Model Summary

| Threat | Current Mitigation | Gap | Severity |
|---|---|---|---|
| Passive network eavesdropping | TLS + AES-256-GCM E2EE | None — well mitigated | — |
| Active MITM on TLS | TOFU (Android), self-signed cert (web) | No cert verification UI on web; TOFU silent on first use | HIGH |
| Server compromise / rogue admin | Zero-knowledge relay (no plaintext ever on server) | None — architecture provides strong protection | — |
| Traffic analysis | 512-byte block padding, random keep-alive intervals | Web client has only one padding size; no cover traffic when idle | MEDIUM |
| Client-side memory forensics | RAM sterilization (Android), tab-blur + about:blank (web) | Web client does not zero-fill CryptoKey objects; browser GC timing is non-deterministic | MEDIUM |
| Brute-force password attack | bcrypt + 10/min rate limit | No account lockout; no password complexity requirements | MEDIUM |
| CSRF | SameSite=Strict cookies | None — well mitigated | — |
| XSS | CSP with strict script-src | `unsafe-inline` in style-src (minor) | LOW |
| Clickjacking | X-Frame-Options DENY | None — well mitigated | — |
| Replay attack within session | AAD with role + sequence number | Sequence numbers are per-session, not per-message-key; no ratchet | LOW |
| Quantum computing | ECDH P-256, | Not post-quantum | LOW (long-term) |

| Threat | Current Mitigation | Gap | Severity |
|---|---|---|---|
| (future) | AES-256 | secure | |

---

# 3. Zero-Knowledge Queue Architecture — Hardening Plan

## 3.1 Queue ID Authorization

**Problem:** Any authenticated user who knows a queue ID can push messages to it. Queue IDs are UUIDs (128-bit entropy), so guessing is infeasible, but IDs can be leaked through logs, referer headers, or browser history.

**Solution: Implement queue ownership verification.**

When `create_queue` is called, the server should store a mapping of `queue_id -> session_id` in server memory (or Redis). When `push_queue` is called, verify that the sender's `request.sid` matches a participant of the target queue. This prevents cross-user queue injection.

```
# In-memory queue ownership (or use Redis for multi-worker)
queue_owners = {}  # queue_id -> set of sid

@socketio.on('create_queue')
def handle_create_queue():
    queue_id = str(uuid.uuid4())
    queue_owners.setdefault(queue_id, set()).add(request.sid)
    join_room(queue_id)
    emit('queue_created', {'queue_id': queue_id})

@socketio.on('push_queue')
def handle_push_queue(data):
    queue_id = data.get('queue_id')
    # Verify sender is a known participant of this queue
    if queue_id not in queue_owners or request.sid not in
queue_owners[queue_id]:
        return  # silently drop
    # ... existing relay logic
```

**Additional hardening:** When a `handshake` payload is received by the host, the host's queue becomes aware of the invitee's queue ID. The server should register bidirectional ownership at this point. Since the server cannot parse the encrypted payload, the client should send an unencrypted `register_peer` event:

```
// After receiving handshake, client sends:
socket.emit('register_peer', {
    my_queue: chatSession.myQueueId,
    peer_queue: chatSession.theirQueueId
});
```

Server-side:

```
@socketio.on('register_peer')
def handle_register_peer(data):
    my_queue = data.get('my_queue')
    peer_queue = data.get('peer_queue')
    if my_queue in queue_owners and request.sid in queue_owners[my_queue]:
```

```
        queue_owners.setdefault(peer_queue, set()).add(request.sid)
```

## 3.2 WebSocket Re-Authentication

**Problem:** Flask sessions can expire while a WebSocket connection remains open. The SocketIO handler checks `session` on `connect`, but the session is not re-validated during the connection lifetime.

**Solution: Implement periodic session validation.**

```
from flask import session

def validate_session():
    """Called periodically or before sensitive operations."""
    if 'username' not in session:
        return False
    return True

# Call in push_queue:
@socketio.on('push_queue')
def handle_push_queue(data):
    if not validate_session():
        emit('session_expired', {})
        disconnect()
        return
    # ... rest of handler
```

Additionally, set an absolute maximum WebSocket session lifetime (e.g., 8 hours) regardless of activity, forcing re-authentication.

## 3.3 Secure Logging Policy

**Problem:** The server currently logs queue IDs and SIDs in debug mode (`app.logger.debug`). Queue IDs are sensitive — they identify active conversations.

**Solution:**

- Never log queue IDs, even in debug mode.
- Use truncated SIDs (first 4 chars) for correlation.
- Add a structured logging configuration that redacts all UUIDs and Base64 strings.

```
import re

def redact_sensitive(log_message):
    """Remove UUIDs and Base64 strings from log messages."""
    log_message = re.sub(r'[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-
9a-f]{12}', '[REDACTED-UUID]', log_message)
    log_message = re.sub(r'[A-Za-z0-9+/]{20,}={0,2}', '[REDACTED-B64]',
log_message)
    return log_message
```

## 3.4 Invite Link Security

**Problem:** Invite links contain the queue ID and public key in the URL hash fragment (`#q=...&k=...`). While hash fragments are not sent to the server, they are stored in browser history and could be leaked through screenshot sharing, sync'd browser history, or shoulder surfing.

**Current mitigations:**

- `history.replaceState(null, null, ' ')` clears the hash after handshake (line 326, `chat.js`).
- Clipboard auto-clear after 30 seconds.
- QR codes use low error correction level (`L`) — harder to photograph and reconstruct.

**Additional improvements:**

- Add a "burn-after-reading" mechanism: the invite link should be single-use. After the invitee connects, the host's queue ID should be rotated, invalidating the original link.
- Implement deep-link handling on Android that clears the link from the system's "Recently Used" apps list.

## 3.5 Password Policy Implementation

**Problem:** No minimum password strength is enforced server-side or client-side.

**Solution — Server-side validation in `/register`:**

```
import re

def validate_password(password):
    if len(password) < 8:
        return {"error": "Password must be at least 8 characters long."}
    categories = 0
    if re.search(r'[A-Z]', password): categories += 1
    if re.search(r'[a-z]', password): categories += 1
    if re.search(r'[0-9]', password): categories += 1
    if re.search(r'[^A-Za-z0-9]', password): categories += 1
    if categories < 3:
        return {"error": "Password must contain characters from at least 3 of:
uppercase, lowercase, digits, special characters."}
    return None

@app.route('/register', methods=['POST'])
@limiter.limit("5 per minute")
def register():
    data = request.get_json()
    if not data:
        return jsonify({"error": "Invalid request"}), 400
    username = data.get('username')
    password = data.get('password')
    if username and len(username) > 50:
        return jsonify({"error": "Username too long"}), 400
    if password and len(password) > 128:
        return jsonify({"error": "Password too long"}), 400
    pwd_error = validate_password(password)
    if pwd_error:
        return jsonify(pwd_error), 400
    # ... existing registration logic
```

**Client-side mirroring in `login.js`:**

```
function validatePassword(p) {
    if (p.length < 8) return 'Password must be at least 8 characters.';
    let cats = 0;
    if (/[A-Z]/.test(p)) cats++;
```

```
    if (/[a-z]/.test(p)) cats++;
    if (/[0-9]/.test(p)) cats++;
    if (/[^A-Za-z0-9]/.test(p)) cats++;
    if (cats < 3) return 'Use 3+ of: uppercase, lowercase, digits, symbols.';
    return null;
}
```

## 3.6 Web Client Key Material Cleanup

**Problem:** When the web client's `resetSession()` is called (panic button, close chat, peer obliviate), the function sets `document.body.innerHTML = ''` and navigates to `about:blank`. However, WebCrypto `CryptoKey` objects are non-extractable and cannot be zero-filled like Android's `ByteArray.fill(0)`. The keys remain in the browser's V8 heap until garbage collected, which is non-deterministic.

**Best-effort mitigation:**

```
async function sanitizeKeys() {
    // Delete all references to trigger GC eligibility
    chatSession.writeKey = null;
    chatSession.readKey = null;
    chatSession.myKeys = null;

    // Force a minor GC trigger by allocating and releasing a large buffer
    // (This is a heuristic — V8's GC is not programmatically triggerable)
    const buf = new ArrayBuffer(16 * 1024 * 1024); // 16MB
    // Let it go out of scope
    setTimeout(() => {}, 0);

    // Navigate away to force page context destruction
    window.location.replace('about:blank');
}
```

Additionally, the `about:blank` navigation after `document.body.innerHTML = ''` already forces the page context to be destroyed, which should cause V8 to release the CryptoKey handles. Browsers in private/incognito mode further reduce the risk of key material persisting in swap or page cache.

**Document this limitation:** Add a note to the user guide that web clients provide weaker key material sanitization compared to Android. For maximum security, the Android app should be preferred for sensitive conversations.

## 3.7 Multi-Worker State Consistency

**Problem:** The in-memory `queue_owners` dictionary and `socket_rate_limits` dictionary are per-process. With multiple eventlet workers, state is not shared.

**Solution:**

- If Redis is configured (`REDIS_URL` is set), use Redis hashes for queue ownership and rate limiting.
- If Redis is not configured, document that the server MUST run with a single worker (which is the default for `socketio.run` without a production WSGI server).
- Add a startup warning when multiple workers are detected without Redis:

```
if not redis_url and os.environ.get('WEB_CONCURRENCY', '1') != '1':
    app.logger.warning("Multi-worker mode without Redis detected. "
                       "Rate limiting and queue ownership will be per-worker. "
                       "Set REDIS_URL for consistent state.")
```

---

# 4. Disappearing Messages — Design & Implementation

## 4.1 Current State Analysis

Disappearing messages are **partially implemented** on both platforms:

**Web Client (`chat.js` lines 164-170):**

```
const timerVal = parseInt(disappearTimerSelect.value, 10);
if (timerVal > 0) {
    setTimeout(() => {
        if (row.parentNode) row.parentNode.removeChild(row);
    }, timerVal * 1000);
}
```

**Android Client (`chat_manager.kt` lines 583-593):**

```
if (disappearTimerSeconds > 0) {
    mainHandler.postDelayed({
        _conversations.update { current ->
            val list = current[chatPartner]?.toMutableList() ?: return@update
current
            list.remove(message)
            current.toMutableMap().apply { put(chatPartner, list) }
        }
    }, disappearTimerSeconds * 1000L)
}
```

Both implementations share the same fundamental flaws:

1. **No peer negotiation** — the timer is set locally and does not inform the other party
2. **No timer enforcement** — nothing prevents a user from not enabling the timer, or from screenshotting before expiry
3. **No timer sync** — messages disappear at different times on each device
4. **DOM-only purging on web** — browser back/forward cache, dev tools, and screen captures can preserve content
5. **No visual countdown** — users cannot see when a message will disappear
6. **Timer applies globally** — all messages in the session use the same timer, not per-message

## 4.2 Design Goals

1. **Peer-negotiated timer** — both parties must agree on the timer duration
2. **Per-message or per-session timer** — flexible selection
3. **Visual countdown** — users see remaining time
4. **Synchronized expiry** — messages disappear at the same wall-clock time on both devices

5. **No server involvement** — timer is enforced client-side, preserving zero-knowledge model
6. **Tamper resistance** — make it difficult (not impossible) to prevent disappearance via dev tools

## 4.3 Protocol Extension

Add a new control message type for timer negotiation:

```
{
  "type": "control",
  "action": "timer_set",
  "duration_seconds": 60,
  "mode": "session"
}
```

Modes: `"off"`, `"session"` (applies to all messages), `"next"` (applies to next message only).

When a client receives `timer_set`, it should:

1. Display a notification: "Peer set messages to disappear in 60 seconds"
2. Apply the timer to all future messages (session mode) or the next message (next mode)
3. Send a `timer_ack` control message back

```
{
  "type": "control",
  "action": "timer_ack",
  "duration_seconds": 60,
  "mode": "session"
}
```

The timer is effective from the moment the message is **displayed** (not sent), ensuring synchronization. Since both clients receive messages at approximately the same time (real-time WebSocket), the visual disappearance will be nearly synchronized.

## 4.4 Implementation — Web Client

**Replace the current `addMessageLine` with:**

```
function addMessageLine(sender, text, disappearAfter) {
    const row = document.createElement('div');
    row.className = 'message' + (sender === 'You' ? ' message-own' : '');

    const senderSpan = document.createElement('span');
    senderSpan.className = 'message-sender';
    senderSpan.textContent = sender + ':';
    row.appendChild(senderSpan);

    const contentNode = document.createTextNode(' ' + text);
    row.appendChild(contentNode);

    // Add countdown timer if applicable
    if (disappearAfter && disappearAfter > 0) {
        const timerSpan = document.createElement('span');
        timerSpan.className = 'message-timer';
        timerSpan.textContent = `${disappearAfter}s`;
        row.appendChild(timerSpan);
```

```
        let remaining = disappearAfter;
        const interval = setInterval(() => {
            remaining--;
            if (remaining <= 0) {
                clearInterval(interval);
                if (row.parentNode) {
                    row.style.transition = 'opacity 0.5s, max-height 0.5s';
                    row.style.opacity = '0';
                    row.style.maxHeight = '0';
                    row.style.overflow = 'hidden';
                    setTimeout(() => {
                        if (row.parentNode) row.parentNode.removeChild(row);
                    }, 500);
                }
            } else {
                timerSpan.textContent = `${remaining}s`;
            }
        }, 1000);
    }

    messagesEl.appendChild(row);
    messagesEl.scrollTop = messagesEl.scrollHeight;
}
```

**Timer state management:**

```
const disappearState = {
    mode: 'off',        // 'off' | 'session' | 'next'
    duration: 0,        // seconds
    nextMessageOnly: false
};

// When user selects a timer:
disappearTimerSelect.addEventListener('change', () => {
    const val = parseInt(disappearTimerSelect.value, 10);
    disappearState.duration = val;
    disappearState.mode = val === 0 ? 'off' : 'session';

    // Notify peer
    if (chatSession.writeKey) {
        const plaintext = JSON.stringify({
            type: 'control',
            action: 'timer_set',
            duration_seconds: val,
            mode: disappearState.mode
        });
        encryptMessage(chatSession.writeKey, plaintext, chatSession.myRole,
chatSession.sendSeq)
            .then(({iv, ciphertext}) => {
                chatSession.sendSeq++;
                socket.emit('push_queue', {
                    queue_id: chatSession.theirQueueId,
                    payload: JSON.stringify({ type: 'message', iv, ciphertext })
                });
            });
    }
});
```

**CSS for disappearing messages:**

```
.message-timer {
    font-size: 0.7em;
    color: #888;
```

```
    margin-left: 8px;
    font-variant-numeric: tabular-nums;
}

.message.expiring {
    animation: pulse-fade 2s ease-in-out infinite;
}

@keyframes pulse-fade {
    0%, 100% { opacity: 1; }
    50% { opacity: 0.6; }
}

.message.fading-out {
    opacity: 0;
    max-height: 0;
    margin: 0;
    padding: 0;
    overflow: hidden;
    transition: all 0.5s ease;
}
```

**Dev tools tamper resistance (best-effort):**

```
// Freeze message nodes to prevent modification via dev tools
Object.freeze(row);
Object.freeze(contentNode);
Object.freeze(timerSpan);

// Detect dev tools opening (heuristic, not foolproof)
const devToolsCheck = new Image();
Object.defineProperty(devToolsCheck, 'id', {
    get: function() {
        // Dev tools detected — optionally trigger panic
        console.warn('Dev tools detected');
    }
});
```

Note: Dev tools detection is inherently unreliable and can be bypassed. The goal is to raise the effort required, not to make it impossible. True security against local compromise requires a different threat model (e.g., hardware-enforced secure enclaves).

## 4.5 Implementation — Android Client

**Replace the current `appendMessage` disappearing logic with:**

```
private fun appendMessage(chatPartner: String, message: ChatMessage) {
    mainHandler.post {
        _conversations.update { currentConversations ->
            val list = currentConversations[chatPartner]?.toMutableList() ?:
mutableListOf()
            list.add(message)
            currentConversations.toMutableMap().apply {
                put(chatPartner, list)
            }
        }

        // Handle disappearing messages with countdown
        if (disappearTimerSeconds > 0 && message.sender != "System") {
            val messageId = System.currentTimeMillis() // unique per message
            startCountdown(chatPartner, message, messageId,
disappearTimerSeconds)
```

```
        }
    }
}

private fun startCountdown(chatPartner: String, message: ChatMessage, messageId:
Long, totalSeconds: Int) {
    var remaining = totalSeconds
    mainHandler.postDelayed(object : Runnable {
        override fun run() {
            if (remaining <= 0) {
                _conversations.update { current ->
                    val list = current[chatPartner]?.toMutableList() ?:
return@update current
                    list.remove(message)
                    current.toMutableMap().apply { put(chatPartner, list) }
                }
                return
            }
            remaining--
            mainHandler.postDelayed(this, 1000)
        }
    }, 1000)
}
```

**Timer negotiation UI in `ChatScreen.kt`:**

Add a `DisappearingMessagesDialog` composable that shows when the user taps the timer button, with options for Off / 15s / 60s / 5min / 30min, and a confirmation that the peer will be notified.

### 4.6 Timer Synchronization Strategy

The key insight is that timer synchronization does not need to be perfect — it needs to be "good enough." Since messages are delivered via WebSocket (typically <100ms latency on LAN, <500ms on internet), starting the countdown from the moment of display on each device independently produces a maximum skew of less than 1 second, which is imperceptible to users.

For scenarios where one party is offline (message queued in SocketIO room, delivered on reconnect), the timer should start from the moment of delivery, not the moment of sending. This is already the natural behavior since the timer is applied in the display logic, not the send logic.

---

## 5. Android App — Improvements & Technical Debt

### 5.1 Replace Standard JCE with Tink

**Problem:** The Android crypto implementation uses raw Java JCE APIs (`KeyPairGenerator`, `KeyAgreement`, `Cipher`). While this works, it is error-prone (manual HKDF implementation, manual AAD construction) and does not benefit from Google's security audits of Tink.

**Solution: Migrate to Google Tink.**

Tink provides:

- Pre-audited implementations of ECDH, AES-GCM, and HKDF

- Automatic key management via `KeysetHandle`
- Consistent API across platforms (Tink also has JavaScript bindings, enabling future cross-platform parity)
- Built-in protection against common pitfalls (IV reuse, insufficient tag length)

```
// build.gradle.kts dependency
implementation("com.google.crypto.tink:tink-android:1.15.0")

// Migration example for key derivation:
import com.google.crypto.tink.KeysetHandle
import com.google.crypto.tink.aead.AeadKeyTemplates
import com.google.crypto.tink.integration.android.AndroidKeysetManager
import com.google.crypto.tink.subtle.EcdhUtil

fun deriveSessionKeysTink(
    myPrivateKey: PrivateKey,
    theirPublicKey: PublicKey,
    myPubKeyBase64: String,
    theirPubKeyBase64: String
): SessionKeys {
    val sharedSecret = EcdhUtil.computeSharedSecret(
        myPrivateKey as ECPublicKey,  // Tink works with raw EC points
        theirPublicKey as ECPublicKey
    )

    // Use Tink's HKDF via subtle primitives
    val clientKey = Hkdf.computeHkdf(
        Hkdf.HmacSha256,
        sharedSecret,
        ByteArray(32),  // salt
        "AnonyMus-Client-To-Server-Key".toByteArray(Charsets.UTF_8),
        32
    )

    val serverKey = Hkdf.computeHkdf(
        Hkdf.HmacSha256,
        sharedSecret,
        ByteArray(32),
        "AnonyMus-Server-To-Client-Key".toByteArray(Charsets.UTF_8),
        32
    )

    val isAlice = myPubKeyBase64 < theirPubKeyBase64
    return if (isAlice) {
        SessionKeys(writeKey = clientKey, readKey = serverKey)
    } else {
        SessionKeys(writeKey = serverKey, readKey = clientKey)
    }
}
```

**Migration strategy:** Implement Tink alongside the existing JCE code behind a `CryptoProvider` interface. Run both implementations in parallel during testing, comparing outputs for correctness. Switch the default to Tink once parity is confirmed.

## 5.2 NSD Discovery Improvements

**Problem:** The current `NsdHelper` discovers `_http._tcp.` services and filters by `APP_NAME` in the service name. This is fragile — it requires the server to advertise via mDNS/Avahi, which it

currently does not. The NSD helper will never find a standard Flask server without an mDNS advertiser.

**Solution: Two-phase LAN discovery.**

**Phase 1 — NSD with custom service type:** Change the service type from `_http._tcp.` to `_anonymus._tcp.` and add an mDNS advertiser to the Python server using `python-zeroconf`:

```
# server.py addition
from zeroconf import Zeroconf, ServiceInfo
import socket

def advertise_mdns(port):
    zeroconf = Zeroconf()
    local_ip = get_local_ip()
    info = ServiceInfo(
        "_anonymus._tcp.local.",
        f"AnonyMus Server._anonymus._tcp.local.",
        addresses=[socket.inet_aton(local_ip)],
        port=port,
        properties={},
    )
    zeroconf.register_service(info)
    return zeroconf
```

**Phase 2 — Fallback IP range scan:** If NSD discovers nothing within 5 seconds, fall back to a rapid TCP connect scan of the local subnet (192.168.x.x or 10.x.x.x) on the configured port. This is faster than waiting for NSD timeout and works even without mDNS support.

```
// NsdHelper.kt addition
fun scanSubnet(port: Int, onFound: (String) -> Unit) {
    val scope = CoroutineScope(Dispatchers.IO)
    scope.launch {
        val localIp = getLocalIpAddress()
        val subnet = localIp.substringBeforeLast('.') + "."
        val deferreds = (1..254).map { host ->
            async {
                try {
                    val socket = java.net.Socket()
                    socket.connect(InetSocketAddress("$subnet$host", port), 200)
                    socket.close()
                    withContext(Dispatchers.Main) { onFound("$subnet$host") }
                } catch (e: Exception) { }
            }
        }
        deferreds.awaitAll()
    }
}
```

## 5.3 Biometric App Lock

**Problem:** The app stores session cookies and server configuration in SharedPreferences. If the device is unlocked, anyone with physical access can open the app and use the active session.

**Solution: Implement biometric authentication as an app lock.**

```
// Using AndroidX Biometric library
implementation("androidx.biometric:biometric:1.1.0")
```

```
// In MainActivity.kt, before any navigation:
val biometricPrompt = BiometricPrompt(
    this,
    ContextCompat.getMainExecutor(this),
    object : BiometricPrompt.AuthenticationCallback() {
        override fun onAuthenticationSucceeded(result:
BiometricPrompt.AuthenticationResult) {
            super.onAuthenticationSucceeded(result)
            // Proceed to app
            setContent { AppNavigation() }
        }
        override fun onAuthenticationFailed() {
            finish()
        }
    }
)

val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Unlock AnonyMus")
    .setSubtitle("Authenticate to access your sessions")
    .setNegativeButtonText("Cancel")
    .build()

biometricPrompt.authenticate(promptInfo)
```

Enable via a toggle in `ConfigScreen.kt`, stored in `PreferencesHelper`.

## 5.4 Secure Screenshot Prevention

**Problem:** Android allows screenshots and screen recording by default, which can capture disappearing messages.

**Solution:**

```
// In MainActivity.kt, onCreate():
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    window.setFlags(
        WindowManager.LayoutParams.FLAG_SECURE,
        WindowManager.LayoutParams.FLAG_SECURE
    )
}
```

This prevents screenshots and screen recording for the entire activity. For a more granular approach, set the flag only on the ChatScreen's window.

## 5.5 Notification Leak Prevention

**Problem:** If notifications are ever added for incoming messages, they could display message content on the lock screen.

**Solution (preemptive):**

- Never display message content in notifications.
- Use a generic notification: "New encrypted message" with no body text.
- Set `setOnlyAlertOnce(true)` to avoid repeated notification sounds.
- Use `NotificationCompat.Builder().setCategory(NotificationCompat.C`

ATEGORY_MESSAGE).setVisibility(NotificationCompat.VISIBILITY_SECRET).

## 5.6 ChatManager Singleton Refactor

**Problem:** `ChatManager` is a Kotlin `object` (singleton) with mutable state. This makes testing difficult and prevents multiple simultaneous connections.

**Solution: Convert to a class with dependency injection.**

```
class ChatManager(
    private val context: Context,
    private val prefs: PreferencesHelper,
    private val cryptoProvider: CryptoProvider
) {
    // ... existing logic, but as instance members instead of static
}

// Provide via Hilt/Dagger or manual DI
class AnonyMusApp : Application() {
    val chatManager by lazy {
        ChatManager(this, PreferencesHelper(this), TinkCryptoProvider())
    }
}
```

## 5.7 Accessibility Improvements

**Problem:** The Compose UI lacks accessibility annotations. Screen readers cannot distinguish between message types, timer states, or action buttons.

**Solution:**

```
// In ChatScreen.kt, message rendering:
Surface(
    // ... existing params
    modifier = Modifier.semantics {
        contentDescription = if (isOwn) "Your message: ${msg.text}" else "Peer
message: ${msg.text}"
        if (!msg.isDecryptedSuccessfully) {
            stateDescription = "Decryption failed"
        }
    }
)

// Button accessibility:
IconButton(
    onClick = { ChatManager.obliviate() },
    modifier = Modifier.semantics {
        contentDescription = "Obliviate: Clear all chat data and notify peer"
    }
)
```

Add `contentDescription` to all IconButtons and interactive elements. Add `stateDescription` for dynamic states (timer value, connection status).

## 5.8 Config Screen — Server URL Validation

**Problem:** The config screen accepts any host/port combination without validation. Users could enter invalid hosts, non-numeric ports, or malicious URLs.

**Solution:**

```
fun isValidServerConfig(host: String, port: String): Boolean {
    val portNum = port.toIntOrNull() ?: return false
    if (portNum < 1 || portNum > 65535) return false
    if (host.isBlank()) return false
    // Reject obviously malicious inputs
    if (host.contains("..") || host.contains("\n") || host.contains("\r"))
return false
    // Validate IP or hostname format
    return host.matches(Regex("^[a-zA-Z0-9._-]+$"))
}
```

---

# 6. Cryptographic Protocol — Analysis & Upgrades

## 6.1 Double Ratchet Consideration

**Current protocol:** Single ECDH key exchange per session, with a monotonically increasing sequence number for AAD. If a session key is compromised, all messages in that session (past and future) can be decrypted.

**Improvement: Implement a simplified Double Ratchet.**

The Signal Protocol's Double Ratchet provides:

- **Forward secrecy per message:** Compromise of a session key reveals only that message and future messages (until the next ratchet step), not past messages.
- **Break-in recovery:** If a key is compromised, the next DH ratchet step generates new keys unknown to the attacker.

However, the full Double Ratchet is complex (requires storing sender/receiver chain keys, DH ratchet step state, skipped message keys). For AnonyMus's 1:1 chat model, a **simplified version** provides 90% of the benefit with 10% of the complexity:

**Simplified Ratchet — Key Derivation Chain:**

Instead of using the same AES-256-GCM key for all messages in a direction, derive each message's key from the previous key using HKDF:

```
K_0 = HKDF(ECDH_shared_secret, "AnonyMus-ChainKey-0")
MK_0 = HKDF(K_0, "AnonyMus-MessageKey-0")
K_1 = HKDF(K_0, "AnonyMus-ChainKey-1")
MK_1 = HKDF(K_1, "AnonyMus-MessageKey-1")
...
```

This means each message uses a unique key, providing per-message forward secrecy. If MK_n is compromised, messages 0 through n-1 remain secure (their keys cannot be derived from MK_n).

```
// Web client addition to crypto.js
async function deriveChainKeys(rootKey) {
    const chainKey = await hkdfDerive(rootKey, new
TextEncoder().encode("AnonyMus-ChainKey"), new Uint8Array(32));
    const messageKey = await hkdfDerive(chainKey, new
TextEncoder().encode("AnonyMus-MessageKey"), new Uint8Array(32));
```

```
    // Import as AES-GCM keys
    const messageKeyObj = await crypto.subtle.importKey(
        'raw', messageKey, { name: 'AES-GCM' }, false, ['encrypt', 'decrypt']
    );
    const nextChainKey = await hkdfDerive(chainKey, new
TextEncoder().encode("AnonyMus-NextChainKey"), new Uint8Array(32));

    return { messageKey: messageKeyObj, nextChainKey };
}
```

**Trade-off:** This adds complexity and a small performance cost (one extra HKDF per message). The sequence number AAD remains valuable for detecting dropped/reordered messages. The recommendation is to implement the simplified chain ratchet but defer the full DH ratchet to a future milestone.

## 6.2 Post-Quantum Readiness

**Current state:** ECDH P-256 and AES-256-GCM are not post-quantum secure. A sufficiently powerful quantum computer could break ECDH (via Shor's algorithm) and weaken AES-256 (via Grover's algorithm, effectively halving the key strength to 128 bits).

**Near-term mitigation:** No action required today. NIST post-quantum standards (ML-KEM/Kyber, ML-DSA/Dilithium) are finalized but not yet widely supported in WebCrypto or Android JCE. The recommended approach is:

1. Design the crypto module behind an abstraction layer (already partially done via `CryptoProvider` interface proposed in 5.1).
2. Monitor WebCrypto and Tink for PQC algorithm support.
3. Plan a hybrid key exchange (ECDH + ML-KEM) for the next major version.

## 6.3 Authenticated Encryption with Associated Data (AEAD) AAD Improvement

**Current AAD:** 5 bytes — 1 byte role ('A' or 'B') + 4 bytes sequence number.

**Improvement:** Expand the AAD to include a session identifier and protocol version, providing defense against cross-protocol attacks if the same keys are accidentally reused in different contexts:

```
function constructAAD(role, seqNum, sessionId, protocolVersion = 2) {
    const encoder = new TextEncoder();
    const roleBytes = encoder.encode(role);
    const sessionBytes = encoder.encode(sessionId); // queue_id pair hash
    const aad = new Uint8Array(1 + 4 + 16 + 1);
    aad[0] = role.charCodeAt(0);
    const view = new DataView(aad.buffer);
    view.setUint32(1, seqNum, false);
    aad.set(sessionBytes.slice(0, 16), 5);  // first 16 bytes of session ID
    aad[21] = protocolVersion;
    return aad;
}
```

This must be implemented as a protocol version bump (v2) with backward compatibility checking.

---

# 7. Performance & Scalability

## 7.1 Current Performance Characteristics

- **Concurrency model:** `eventlet` monkey-patching provides async I/O for WebSocket handling. This is adequate for small-scale deployments (10-50 concurrent users) but does not scale to thousands of connections.
- **Database:** SQLite without WAL mode — single writer, blocking reads during writes. PostgreSQL is supported but not documented as the recommended production backend.
- **Memory:** All in-memory state (rate limits, queue ownership) is per-process. No shared state across workers.
- **Payload size:** 100KB WebSocket payload limit, 1MB HTTP request limit. The 512-byte padding means each encrypted message is ~524+ bytes (512 padded plaintext + 12 IV + 16 GCM tag + Base64 overhead ≈ 864 bytes on wire).

## 7.2 SQLite WAL Mode

**Immediate fix with high impact:**

```
def get_connection():
    if DATABASE_URL:
        import psycopg2
        return psycopg2.connect(DATABASE_URL)
    else:
        conn = sqlite3.connect(DB_FILE)
        conn.execute('PRAGMA journal_mode=WAL')
        conn.execute('PRAGMA busy_timeout=5000')
        return conn
```

WAL mode allows concurrent readers while a writer is active, eliminating `database is locked` errors under moderate load. The `busy_timeout` pragma causes SQLite to retry for up to 5 seconds before raising an error, providing additional resilience against brief write contention.

## 7.3 Connection Pooling

**Problem:** Each database operation in `database.py` opens a new connection and closes it. This is wasteful for SQLite (WAL mode benefits from connection reuse) and critical for PostgreSQL (connection setup is expensive).

**Solution:**

```
import sqlite3
from contextlib import contextmanager

_connection_pool = None

def get_connection():
    global _connection_pool
    if DATABASE_URL:
        import psycopg2
        from psycopg2 import pool
        if _connection_pool is None:
            _connection_pool = pool.ThreadedConnectionPool(
                minconn=2, maxconn=10, dsn=DATABASE_URL
```

```
            )
        return _connection_pool.getconn()
    else:
        conn = sqlite3.connect(DB_FILE, check_same_thread=False)
        conn.execute('PRAGMA journal_mode=WAL')
        conn.execute('PRAGMA busy_timeout=5000')
        return conn

def release_connection(conn):
    if DATABASE_URL and _connection_pool:
        _connection_pool.putconn(conn)
    else:
        conn.close()
```

## 7.4 Production WSGI/ASGI Server

**Problem:** `socketio.run(app)` uses Flask's development server, which is single-threaded and not suitable for production.

**Solution: Use Gunicorn with eventlet worker:**

```
gunicorn --worker-class eventlet --workers 1 --bind 0.0.0.0:5000 --keyfile
key.pem --certfile cert.pem server:app
```

For higher concurrency, use multiple workers with Redis as the message queue:

```
gunicorn --worker-class eventlet --workers 4 --bind 0.0.0.0:5000 --keyfile
key.pem --certfile cert.pem server:app
```

Note: With multiple workers, queue ownership and rate limiting must use Redis (see 3.5).

## 7.5 WebSocket Compression

**Problem:** The 512-byte block padding means most messages are padded to 512 bytes regardless of content length. A short "hello" message becomes ~864 bytes on wire (with Base64 overhead). WebSocket compression (permessage-deflate) can reduce this by 30-50%.

**Solution: Enable permessage-deflate on the server:**

```
socketio = SocketIO(
    app,
    cors_allowed_origins=allowed_origins,
    message_queue=redis_url,
    transports=['websocket'],
    engineio_logger=False,
    ping_timeout=60,
    ping_interval=25,
)
```

Socket.IO's `engineio` automatically negotiates `permessage-deflate` with compatible clients. No additional code is needed on the server. The browser's Socket.IO client and the Android `socket.io-client` both support this by default.

## 7.6 Keep-Alive Interval Optimization

**Current:** Web client sends heartbeat every 2-7 seconds (random). Android sends "static" every 2-7 seconds.

**Problem:** This is excessive for battery-powered mobile devices and generates significant traffic overhead for idle sessions.

**Recommendation:**

- Web client: 10-30 second random interval (still provides traffic obfuscation but reduces overhead by 5x).
- Android client: 15-45 second random interval, with adaptive scaling — increase interval when the device is in power-saving mode or Doze mode.

```
// Web: adjusted keep-alive
staticInterval = setTimeout(sendKeepAlive, Math.random() * 20000 + 10000); //
10-30s

// Android: adaptive keep-alive
val powerManager = context.getSystemService(Context.POWER_SERVICE) as
PowerManager
val baseInterval = if (powerManager.isPowerSaveMode) 60000L else 15000L
val jitter = (Math.random() * 30000).toLong()
mainHandler.postDelayed(this, baseInterval + jitter)
```

---

# 8. Deployment, DevOps & Infrastructure

## 8.1 Docker Containerization

**Current state:** No containerization. Deployment is via `pip install -r requirements.txt && python server.py`.

**Solution:**

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY server.py database.py ./
COPY templates/ ./templates/
COPY static/ ./static/

# Create non-root user
RUN useradd -m -r anonymus && chown -R anonymus:anonymus /app
USER anonymus

# Environment variables
ENV FLASK_DEBUG=false
ENV PORT=5000
ENV DISABLE_SSL=false

EXPOSE 5000

CMD ["gunicorn", "--worker-class", "eventlet", "--workers", "1", \
     "--bind", "0.0.0.0:5000", \
```

```
    "--keyfile", "key.pem", "--certfile", "cert.pem", \
    "server:app"]

# docker-compose.yml
version: '3.8'
services:
  anonymus-server:
    build: .
    ports:
      - "5000:5000"
    environment:
      - FLASK_SECRET_KEY=${FLASK_SECRET_KEY}
      - REDIS_URL=redis://redis:6379
      - CORS_ORIGINS=https://yourdomain.com
    volumes:
      - ./data:/app/data  # SQLite DB persistence
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data
    command: redis-server --appendonly yes

volumes:
  redis-data:
```

## 8.2 Tor Hidden Service Deployment

The repository includes `tor_setup.md` describing Tor hidden service deployment. This is a critical feature for AnonyMus's threat model — it allows the server to be accessed without revealing its IP address.

**Recommended improvements to Tor deployment:**

1. **Use Ephemeral Onion Services** — no persistent `.onion` address, generated fresh each deployment:

   ```
   # torrc
   HiddenServiceDir /tmp/tor-anonymus/
   HiddenServicePort 80 127.0.0.1:5000
   HiddenServiceVersion 3
   ```

2. **Client-side Tor Proxy** — The Android app should support routing through Orbot (Tor proxy on Android). Add a setting in `ConfigScreen.kt`:

   ```
   // Proxy configuration for OkHttp
   val proxy = java.net.Proxy(
       java.net.Proxy.Type.HTTP,
       java.net.InetSocketAddress("127.0.0.1", 9050)  // Orbot's default
   SOCKS port
   )
   builder.proxy(proxy)
   // Note: Socket.IO over SOCKS requires additional configuration
   ```

3. **V3 Onion Address Authentication** — Tor V3 supports client authorization via `HidServAuth`. Implement this as an optional server configuration to prevent unauthorized

clients from discovering the onion service.

## 8.3 SSL Certificate Management

**Current state:** Self-signed certificates with 1-year validity, auto-generated on first run.

**Improvements:**

1. **Let's Encrypt integration** for public deployments:

```
certbot certonly --standalone -d anonymus.yourdomain.com
```

2. **Certificate pinning on web client** — currently only Android has TOFU pinning. Add a web equivalent using the `Certificate` API:

```
// On first connection, store the certificate fingerprint
async function pinCertificate() {
    const response = await fetch('/certificate-fingerprint');
    const fingerprint = await response.text();
    localStorage.setItem('server_cert_fp', fingerprint);
}

// On subsequent connections, verify
// Note: Web Crypto cannot directly inspect TLS certificates.
// Alternative: serve a signed token that includes the cert fingerprint.
```

3. **Certificate rotation notification** — when the server's certificate changes (e.g., renewal), Android clients should detect the change and prompt the user rather than silently failing or accepting.

## 8.4 Health Check Endpoint

Add a `/health` endpoint for container orchestration monitoring:

```
@app.route('/health', methods=['GET'])
def health():
    return jsonify({"status": "ok", "timestamp":
datetime.datetime.utcnow().isoformat()}), 200
```

This endpoint should not require authentication (it reveals no sensitive information) and should be excluded from rate limiting.

---

# 9. Testing Strategy

## 9.1 Current Test Coverage

The repository includes:

- `tests/test_integration.py` — integration tests for the server
- `tests/test_database.py` — database operation tests
- `tests/test_crypto.js` — WebCrypto unit tests

## 9.2 Recommended Test Expansion

**Unit Tests (priority: HIGH):**

| Component | Test Cases | Framework |
|---|---|---|
| `crypto.js` | Key generation, encryption/decryption roundtrip, AAD validation, sequence number rejection, padding roundtrip, safety number determinism | Existing `test_crypto.js` — expand |
| `CryptoUtils.kt` | Same as web crypto tests — ensure cross-platform parity | JUnit 5 |
| `database.py` | Registration, duplicate registration, login success, login failure, timing attack constant-time, SQL injection attempt | pytest |
| `server.py` | Rate limiting enforcement, payload size rejection, session validation, queue ownership | pytest + pytest-socketio |

**Integration Tests (priority: HIGH):**

| Test Case | Description |
|---|---|
| Full handshake flow | Alice creates queue, generates invite, Bob joins, both derive matching safety numbers and session keys |
| Message roundtrip | Alice sends message, Bob receives and decrypts correctly |
| Disappearing messages | Timer-set message disappears on both clients |
| Reconnection | Alice disconnects, reconnects, sends queue_update, Bob receives |
| Panic button | Alice triggers obliviate, Bob receives control message and resets |
| Cross-platform | Web client and Android client communicate successfully |
| Offline delivery | Alice sends message while Bob is offline, Bob connects and receives |

**Security Tests (priority: MEDIUM):**

| Test Case | Description |
|---|---|
| AAD tampering | Modify AAD bytes in transit — verify decryption fails |
| Sequence number replay | Replay a captured encrypted message — verify |

| Test Case | Description |
|---|---|
| | rejection |
| IV reuse detection | Send two messages with same IV (should never happen with random IVs) |
| Padding oracle | Verify that padding removal does not leak information about plaintext length |
| Certificate pinning | Connect to server with different cert — verify Android rejects |

**End-to-End Tests (priority: MEDIUM):**

Use Playwright for web, Espresso for Android:

```
// Playwright E2E example
test('complete chat flow', async ({ page: alice, page: bob }) => {
    // Alice logs in and creates invite
    await alice.goto('/login');
    // ... fill credentials, submit
    await alice.waitForSelector('#invite-link-display');
    const inviteLink = await alice.textContent('#invite-link-display');

    // Bob opens invite link
    await bob.goto(inviteLink);
    await bob.click('#btn-accept-invite');

    // Alice sends message
    await alice.fill('#message-input', 'Hello Bob');
    await alice.click('#message-form button[type="submit"]');

    // Bob receives message
    await bob.waitForSelector('.message:last-child');
    const messages = await bob.textContent('.message:last-child');
    expect(messages).toContain('Hello Bob');
});
```

## 9.3 CI/CD Pipeline

```
# .github/workflows/test.yml
name: Test
on: [push, pull_request]
jobs:
  test-python:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pip install pytest pytest-socketio
      - run: pytest tests/

  test-javascript:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
```

```
      with:
        node-version: '20'
    - run: node tests/test_crypto.js  # or use a test runner

test-android:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '17'
    - run: cd AnonyMus_android && ./gradlew test
```

---

# 10. Implementation Roadmap

## Phase 1: Foundation Hardening (Weeks 1-3)

| Task | Priority | Effort | Files |
|---|---|---|---|
| Fix duplicate `const socket` in `chat.js` | HIGH | 0.5h | `chat.js` |
| Enable SQLite WAL mode + busy_timeout | HIGH | 1h | `database.py` |
| Add queue ownership verification | HIGH | 4h | `server.py` |
| Add username character validation | MEDIUM | 2h | `server.py`, `login.js` |
| Add password policy (server + client) | HIGH | 2h | `server.py`, `login.js` |
| Remove or fully implement `device_id` | MEDIUM | 3h | `chat_manager.kt`, `database.py`, `server.py` |
| Add web client key sanitization | MEDIUM | 1h | `chat.js` |
| Add `/health` endpoint | LOW | 0.5h | `server.py` |
| Restrict CORS from wildcard default | HIGH | 1h | `server.py` |
| Add Dockerfile + docker-compose.yml | MEDIUM | 3h | New files |
| Expand test coverage (unit + integration) | HIGH | 8h | `tests/` |

## Phase 2: Disappearing Messages (Weeks 4-5)

| Task | Priority | Effort | Files |
|---|---|---|---|
| Design timer negotiation protocol | HIGH | 2h | New spec |
| Implement `timer_set`/`timer_ack` control messages | HIGH | 4h | `chat.js`, `chat_manager.kt` |
| Implement visual countdown (web) | HIGH | 4h | `chat.js`, `style.css` |
| Implement visual countdown (Android) | HIGH | 4h | `chat_screen.kt` |
| Add timer option UI (web dropdown expansion) | MEDIUM | 2h | `chat.html`, `chat.js` |
| Add timer negotiation UI (Android dialog) | MEDIUM | 3h | `chat_screen.kt` |
| Add fade-out CSS animation | LOW | 1h | `style.css` |
| Integration test for disappearing messages | HIGH | 2h | `tests/` |

## Phase 3: Android App Improvements (Weeks 6-8)

| Task | Priority | Effort | Files |
|---|---|---|---|
| Migrate crypto to Tink behind interface | HIGH | 8h | `crypto_utils.kt`, new `CryptoProvider.kt` |
| Improve NSD discovery (custom type + subnet scan) | MEDIUM | 6h | `nsd_helper.kt`, `config_screen.kt` |
| Add biometric app lock | MEDIUM | 4h | `main_activity.kt`, `config_screen.kt`, `preferences_helper.kt` |
| Add FLAG_SECURE for screenshot prevention | MEDIUM | 1h | `main_activity.kt` |
| Refactor ChatManager from singleton to class | MEDIUM | 6h | `chat_manager.kt`, all UI files |
| Add accessibility | MEDIUM | 4h | All `*_screen.kt` |

| Task | Priority | Effort | Files |
|---|---|---|---|
| annotations to Compose UI | | | files |
| Add server URL validation | LOW | 2h | `config_screen.kt` |
| Add TOFU certificate display + reset | HIGH | 3h | `chat_manager.kt`, `config_screen.kt` |

## Phase 4: Cryptographic Upgrades (Weeks 9-10)

| Task | Priority | Effort | Files |
|---|---|---|---|
| Implement simplified chain ratchet | HIGH | 8h | `crypto.js`, `crypto_utils.kt` |
| Expand AAD with session ID + protocol version | MEDIUM | 4h | `crypto.js`, `crypto_utils.kt` |
| Cross-platform crypto parity test suite | HIGH | 4h | New test files |
| Add invite link burn-after-reading | MEDIUM | 4h | `chat.js`, `server.py`, `chat_manager.kt` |
| WebSocket re-authentication check | MEDIUM | 2h | `server.py` |

## Phase 5: DevOps & Polish (Weeks 11-12)

| Task | Priority | Effort | Files |
|---|---|---|---|
| Add mDNS advertiser to Python server | MEDIUM | 3h | `server.py`, `requirements.txt` |
| Set up CI/CD pipeline (GitHub Actions) | MEDIUM | 4h | `.github/workflows/` |
| Add secure logging policy | MEDIUM | 2h | `server.py` |
| Optimize keep-alive intervals | LOW | 1h | `chat.js`, `chat_manager.kt` |
| Write deployment documentation updates | LOW | 3h | `SETUP.md`, `deployment_guide.md` |
| Performance testing under load | MEDIUM | 4h | New test scripts |

## 11. Risk Register

| ID | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| R1 | WebCrypto API unavailable (non-HTTPS context) | LOW | HIGH | Server enforces HTTPS; client checks `window.crypto.subtle` on connect |
| R2 | SQLite corruption under concurrent write load | MEDIUM | HIGH | Enable WAL mode + busy_timeout; recommend PostgreSQL for production |
| R3 | Browser extension bypasses CSP and reads messages from DOM | LOW | CRITICAL | Disappearing messages reduce window; document that client-side compromise is out of scope |
| R4 | Self-signed cert TOFU accepts MITM on first connection | HIGH | HIGH | Add fingerprint display on both platforms; add cert change detection on Android |
| R5 | Multi-worker deployment without Redis causes state inconsistency | MEDIUM | HIGH | Add startup warning; document single-worker requirement without Redis |
| R6 | Chain ratchet implementation introduces cross-platform parity bug | MEDIUM | HIGH | Extensive cross-platform test suite; feature flag for gradual rollout |
| R7 | Android Tink migration breaks existing sessions | LOW | MEDIUM | Backend interface; parallel running period; versioned protocol |
| R8 | Tor hidden service performance degrades user experience | MEDIUM | LOW | Document expected latency; optimize WebSocket |

| ID | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| | | | | ping/timeout for high-latency connections |
| R9 | Invite link leaked via browser sync or screenshot | MEDIUM | MEDIUM | Burn-after-reading; hash fragment (not sent to server); clipboard auto-clear |
| R10 | Disappearing messages give false sense of security | HIGH | MEDIUM | Document limitations clearly; add visual warnings that screenshots are possible |

---

# 12. Success Metrics & KPIs

## 12.1 Security Metrics

| Metric | Current State | Target | Measurement Method |
|---|---|---|---|
| Test coverage (lines) | Unknown (~15% est.) | >80% | pytest-cov / JaCoCo |
| Known CVEs in dependencies | Unknown | 0 critical/high | `pip audit` / `gradle dependencyCheckAnalyze` |
| Time to patch critical vulnerability | Unknown | <48 hours | Tracking from CVE disclosure to deployed fix |
| Cryptographic agility (time to swap algorithm) | N/A | <1 week | Drill exercise |
| Cross-platform crypto test pass rate | N/A | 100% | CI pipeline |

## 12.2 Privacy Metrics

| Metric | Current State | Target | Measurement Method |
|---|---|---|---|
| Server-side plaintext exposure points | 0 | 0 | Code audit |
| Message metadata stored server-side | Username + timestamp (session) | Username only | Code audit |

| Metric | Current State | Target | Measurement Method |
|---|---|---|---|
| Data persisted on client after panic reset | Session cookie (Android) | 0 | Manual verification |
| Disappearing message effectiveness | N/A (not fully implemented) | Messages removed within 1s of timer | Automated test |

### 12.3 Performance Metrics

| Metric | Current State | Target | Measurement Method |
|---|---|---|---|
| Message delivery latency (LAN) | <50ms | <50ms | Integration test with timestamps |
| Message delivery latency (internet) | Unknown | <500ms | Integration test |
| Concurrent connections supported | ~50 (single worker) | 200+ (with Redis + multi-worker) | Load test (locust/k6) |
| Database operations under load | Unknown (no WAL) | <10ms p99 | Load test |
| Android battery drain (idle session) | Unknown | <2%/hour | Profile with Android Studio |

### 12.4 Usability Metrics

| Metric | Current State | Target | Measurement Method |
|---|---|---|---|
| Steps to start encrypted chat (web) | 4 (login -> create -> share -> accept) | 4 (same, but more polished) | User testing |
| Steps to start encrypted chat (Android) | 5 (config -> login -> connect -> share -> accept) | 4 (auto-discover via NSD) | User testing |
| Safety number verification completion rate | ~0% (no UI prompt) | >30% | Analytics (opt-in) |
| Accessibility score (Android) | Unknown | WCAG 2.1 AA | Accessibility Scanner |

---

## Appendix A: File Inventory & Line Counts

| File | Lines | Purpose |
|---|---|---|
| `server.py` | 324 | Flask + SocketIO server, HTTP routes, WebSocket handlers, SSL |
| `database.py` | 103 | SQLite/PostgreSQL user |

| File | Lines | Purpose |
|---|---|---|
| | | management with bcrypt |
| `static/crypto.js` | 212 | ECDH P-256, HKDF, AES-256-GCM, padding, safety numbers |
| `static/chat.js` | 459 | WebSocket client, session management, disappearing messages, panic |
| `static/login.js` | 61 | Login/registration UI logic |
| `static/style.css` | ~300 (est.) | Chat UI styling |
| `templates/chat.html` | ~200 (est.) | Chat page template |
| `templates/login.html` | ~100 (est.) | Login/registration page template |
| `requirements.txt` | ~10 | Python dependencies |
| `crypto_utils.kt` | 222 | Android ECDH + AES-GCM via Java JCE |
| `chat_manager.kt` | 602 | Android WebSocket, TOFU, keep-alive, disappearing messages |
| `nsd_helper.kt` | 74 | Android LAN server discovery |
| `navigation.kt` | 42 | Jetpack Compose navigation graph |
| `chat_screen.kt` | 199 | Compose chat UI with covert mode, timer, panic |
| `setup_screen.kt` | ~100 (est.) | Invite link generation/acceptance |
| `auth_screen.kt` | ~80 (est.) | Login/registration forms |
| `config_screen.kt` | ~80 (est.) | Server configuration |
| `preferences_helper.kt` | ~60 (est.) | SharedPreferences wrapper |
| `main_activity.kt` | ~30 (est.) | Activity entry point |
| **Total** | **~3,300** | |

# Appendix B: Dependency Audit

## Python (requirements.txt)

| Package | Purpose | Risk Assessment |
|---|---|---|
| Flask | Web framework | Mature, well-maintained |
| Flask-SocketIO | WebSocket support | Mature, actively maintained |
| eventlet | Async I/O | Stable, but consider migrating to gevent for better |

| Package | Purpose | Risk Assessment |
|---|---|---|
| | | performance |
| python-dotenv | Environment variable loading | Mature, no risk |
| bcrypt | Password hashing | Mature, audited |
| cryptography | X.509 certificate generation | Mature, audited by Google |
| flask-limiter | HTTP rate limiting | Mature, supports Redis backend |
| psycopg2 (optional) | PostgreSQL driver | Mature |

## Android (gradle/libs.versions.toml)

| Package | Purpose | Risk Assessment |
|---|---|---|
| Socket.IO client | WebSocket communication | Mature |
| OkHttp | HTTP + TLS | Mature, audited by Square |
| Jetpack Compose | UI framework | First-party Google, actively developed |
| Material3 | Design system | First-party Google |
| Navigation Compose | Navigation | First-party Google |

## Web Client

| Dependency | Purpose | Risk Assessment |
|---|---|---|
| Socket.IO client (CDN) | WebSocket communication | Mature, loaded from CDN (in CSP allowlist) |
| QRCode.js | QR code generation | Lightweight, no known CVEs |
| WebCrypto API | Cryptography | Browser-native, no dependency |

---

*End of Plan*